

Nike Global Catalog Data Pipeline

1 Project Overview and Goals

This project focused on building an end-to-end cloud-based data analytics pipeline using the Nike Global Catalogue 2026 dataset. The goal was to design a scalable system that ingests raw data, processes and transforms it, stores it in a data warehouse, and presents insights through an interactive dashboard.

The primary objectives of the project were:

- Ingest and unify 45 country-level CSV files into a single dataset
- Normalize all product prices into USD for cross-market comparison
- Store processed data in a scalable warehouse (BigQuery)
- Develop an interactive analytics dashboard using Looker Studio

This pipeline demonstrates how large-scale retail data can be transformed into actionable insights using modern cloud technologies.

2 Dataset Description

The dataset used in this project is the *Nike Global Catalogue 2026* dataset sourced from [Kaggle](#). This dataset represents Nike's global e-commerce catalog and includes pricing, discounts, inventory status, and product metadata across multiple international markets. The dataset is designed for large-scale retail analytics and machine learning workflows, making it well-suited for a cloud-based data pipeline. Each country has its own CSV file with 35 columns each. Some of these key columns are: `country_code`, `gender_segment`, `category`, `price_local`, `discount_pct`, `product_name`, and `in_stock`.

3 Pipeline Workflow and Architecture

The pipeline was built entirely on Google Cloud Platform (GCP) and consists of six main stages:

1. Data Ingestion

- Source: Kaggle dataset (45 CSV files)
- Uploaded to Google Cloud Storage (GCS) raw bucket

2. Storage (Raw Layer)

- GCS Bucket: nike-pipeline-raw
- Stores original CSV files without modification

3. Data Processing (Dataproc + PySpark)

- A Dataproc cluster runs a PySpark job to:
 - Combine all country files into one dataset
 - Clean malformed data
 - Normalize currency values to USD
 - Fix data inconsistencies

4. Storage (Processed Layer)

- Cleaned data written to GCS in Parquet format
- GCS Bucket: nike-pipeline-processed

5. Data Warehouse (BigQuery)

- Parquet files loaded into BigQuery
- Analytical SQL views created for reporting

6. Visualization (Looker Studio)

- Connected to BigQuery
- Interactive dashboards built for analysis

4 Data Processing Steps

The PySpark transformation job performed several critical data cleaning and transformation operations:

Data Cleaning

- Handled malformed CSV rows caused by unescaped quotes using DROPMALFORMED mode
- Fixed incorrect schema inference (e.g., price_local incorrectly read as string) by defining an explicit schema

Transformations

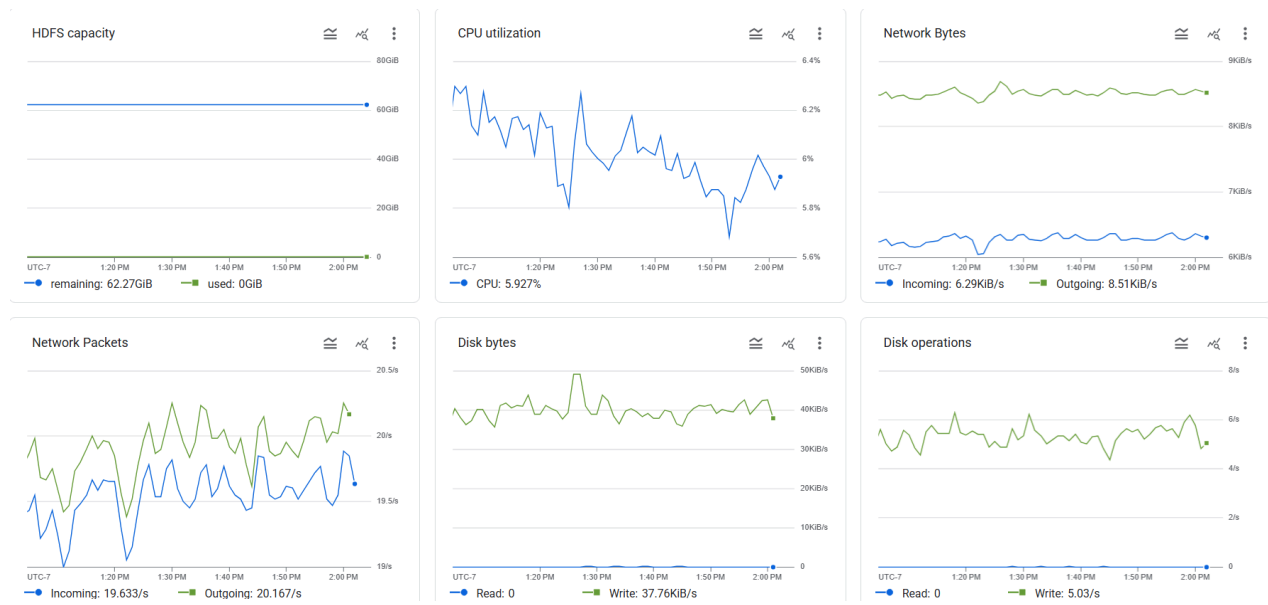
- Converted all price_local values to USD using exchange rates
- Normalized negative discount_pct values using absolute value
- Combined all 45 datasets into a unified dataset
- Dropped columns with all null or identical values (e.g., size_count, employee_price, record_source)

Schema & Structural Fixes

- Removed problematic partitioning due to missing country_code in Parquet output
- Ensured consistent schema across all records

Intermediate Outputs

- Cleaned dataset written as Parquet files to GCS
- Parquet chosen for:
 - Columnar storage efficiency
 - Faster querying in BigQuery
 - Reduced storage size



DataProc Cluster Details

5 Results Obtained

After processing, the pipeline produced:

Final Dataset

- Unified dataset of ~1.5 million rows
- Fully cleaned and normalized
- Stored in BigQuery

Analytical Views

- 5 SQL views created to support analytics

Field name	Type
country_code	STRING
category	STRING
gender_segment	STRING
availability_level	STRING
total_skus	INTEGER
in_stock_count	INTEGER
out_of_stock_count	INTEGER
in_stock_pct	FLOAT
avail_high	INTEGER
avail_low	INTEGER
avail_oos	INTEGER

Availability View Schema

Field name	Type
country_code	STRING
category	STRING
gender_segment	STRING
total_products	INTEGER
discounted_products	INTEGER
pct_products_discounted	FLOAT
avg_discount_pct	FLOAT
max_discount_pct	FLOAT
band_no_discount	INTEGER
band_1_to_10	INTEGER
band_11_to_25	INTEGER
band_26_to_50	INTEGER
band_over_50	INTEGER

Discount View Schema

Field name	Type
country_code	STRING
category	STRING
subcategory	STRING
gender_segment	STRING
unique_products	INTEGER
total_skus	INTEGER
avg_price_usd	FLOAT
discounted_skus	INTEGER
pct_discounted	FLOAT

Category Coverage View Schema

Field name	Type
country_code	STRING
currency	STRING
category	STRING
product_count	INTEGER
avg_price_local	FLOAT
min_price_local	FLOAT
max_price_local	FLOAT
avg_price_usd	FLOAT
min_price_usd	FLOAT
max_price_usd	FLOAT
price_spread_usd	FLOAT

Price View Schema

Field name	Type
country_code	STRING
category	STRING
product_name	STRING
model_number	STRING
gender_segment	STRING
sku_count	INTEGER
avg_price_usd	FLOAT
avg_discount_pct	FLOAT
ever_discounted	INTEGER
in_stock_skus	INTEGER
rank_in_category	INTEGER

Top Products View Schema

Key Metrics Computed

- Global average price: **\$67.58**
- Discount coverage: ~392K discounted SKUs
- In-stock rate: **97.1%**

Insights Generated

- Turkey has the highest average prices
- Korea is Nike's most aggressive discounting market, with the largest share of products receiving 26 to 50% off
- Footwear consistently receives deeper discounts than apparel, particularly across European markets, but also holds the most expensive products
- The US has the largest product assortment with over 8,500 unique apparel products

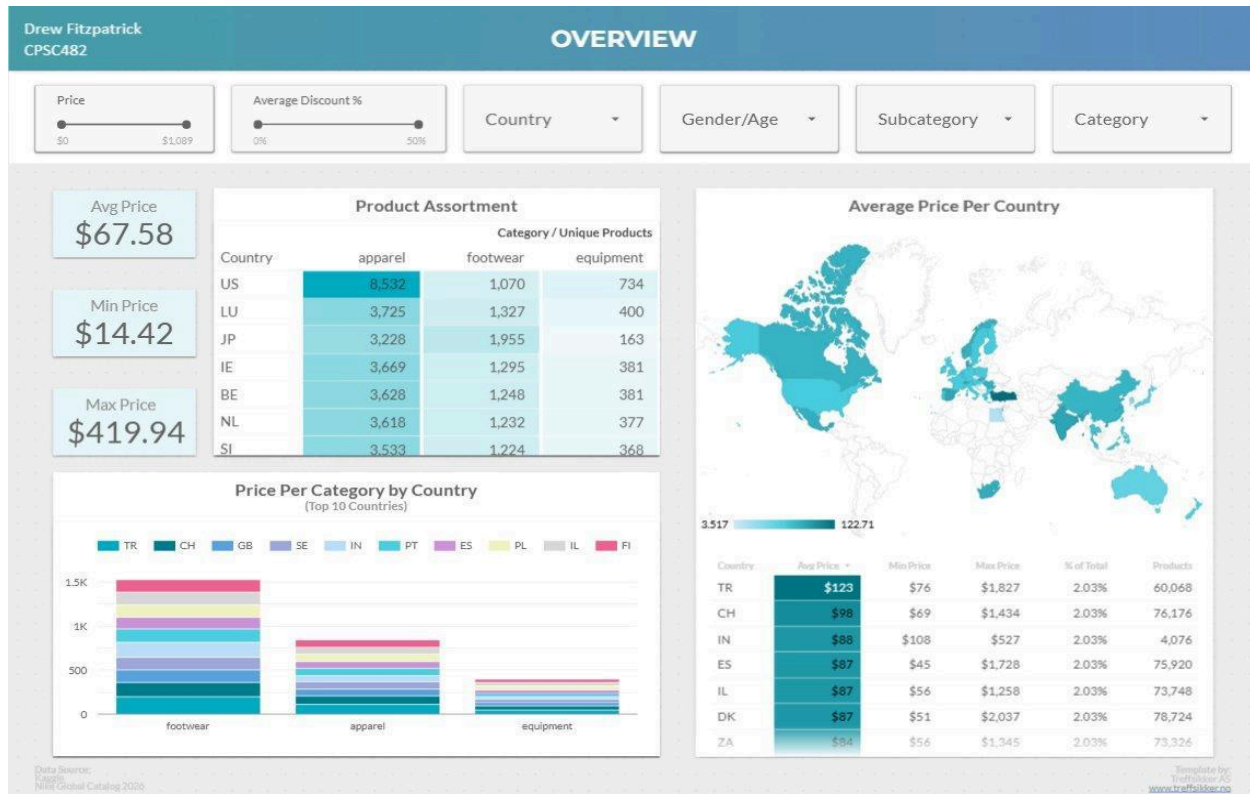
6 Visualizations / Dashboard

An interactive dashboard was built in Looker Studio with multiple pages:

Overview Dashboard

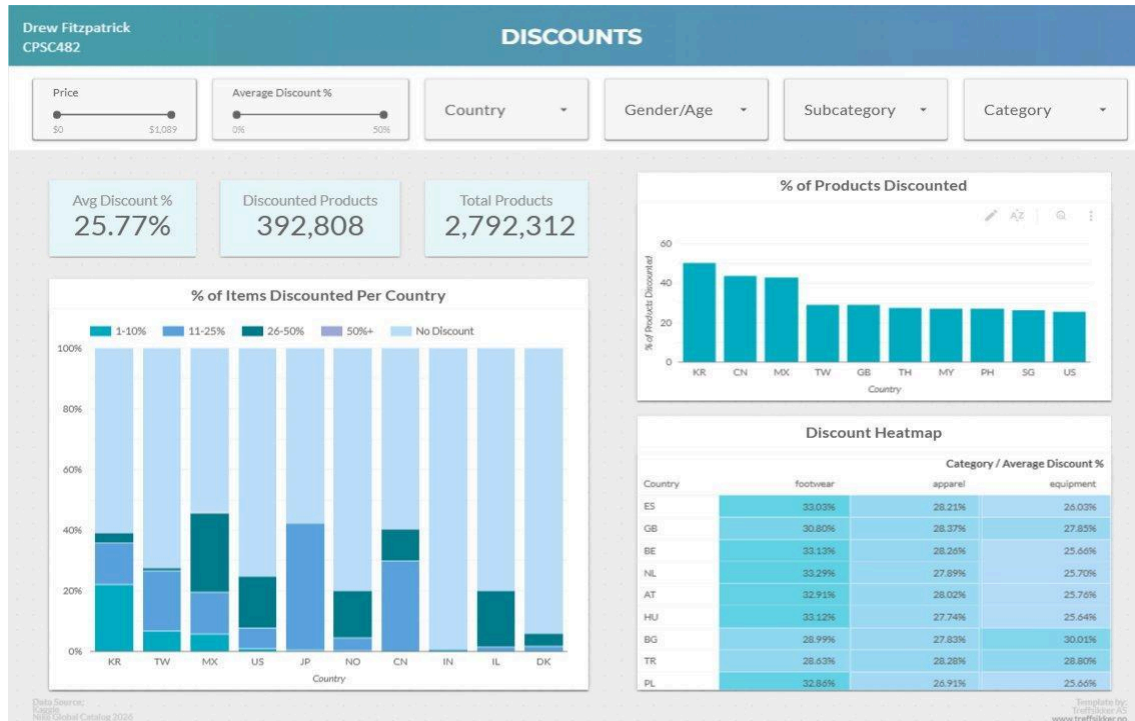
- Global price statistics (avg, min, max)

- Geo map of prices by country
- Product assortment matrix
- Category-level price comparisons



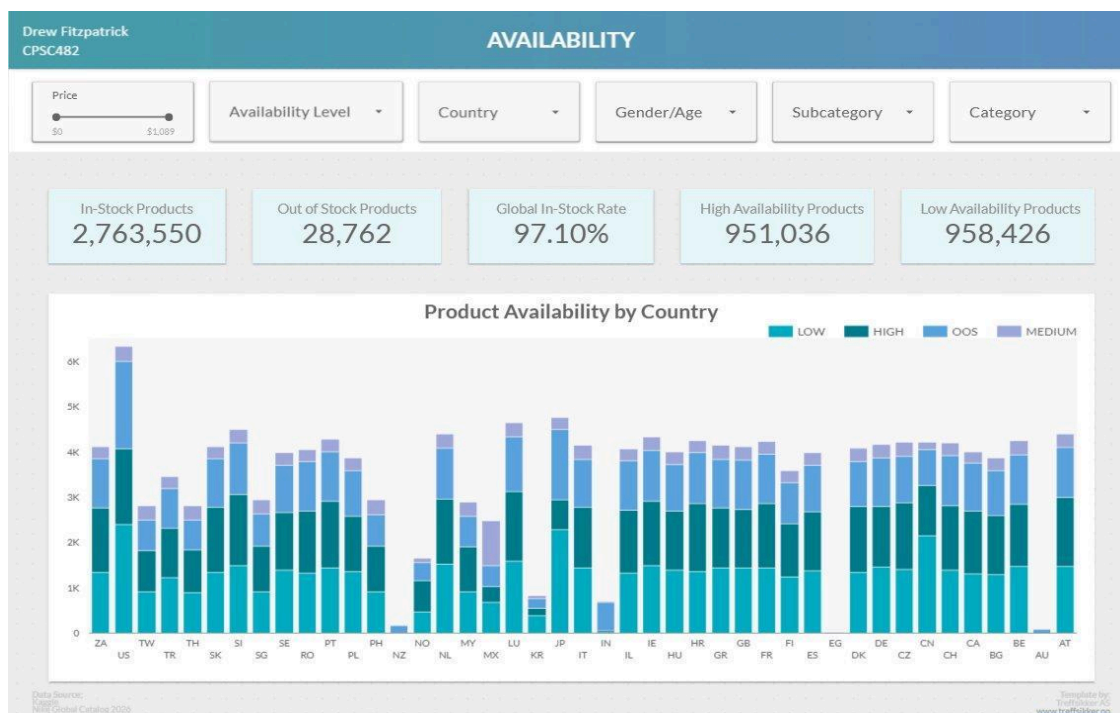
Discounts Dashboard

- Average discount percentage (~25.77%)
- Discount distribution by country
- Heatmaps of discount intensity by category



Availability Dashboard

- Global in-stock rate (97.1%)
- Inventory breakdown (HIGH / LOW / OOS)
- Country-level stock comparisons



Features

- Interactive filters:
 - Country
 - Category
 - Gender
 - Price range slider
 - Average discount % slider

These dashboards allow users to explore cross-market retail patterns dynamically.

7 Challenges Encountered and Resolutions

1. Dataproc Cluster Creation Failure

- Issue: Cluster stuck in error state
- Cause: *internallpOnly* = *true* configuration
- Solution: Recreated cluster without restriction

2. Malformed CSV Rows

- Issue: Data misalignment due to unescaped quotes
- Solution: Used DROPMALFORMED mode in Spark

3. Incorrect Data Types

- Issue: *price_local* read as string
- Solution: Defined explicit schema in PySpark

4. Negative Discount Values

- Issue: Invalid negative percentages
- Solution: Applied absolute value transformation

5. Partitioning Errors

- Issue: Missing *country_code* in Parquet output
- Solution: Removed *partitionBy()* during write

8 Lessons Learned

This project provided valuable hands-on experience in building a real-world, end-to-end data pipeline and highlighted several important lessons about working with large-scale data systems. One of the most significant takeaways was the importance of defining and enforcing schemas when using distributed processing frameworks like PySpark, as incorrect type inference can easily lead to downstream errors. The project also emphasized that data quality issues such as malformed rows, inconsistent formats, and invalid values are common in real-world datasets and must be handled proactively during the cleaning phase. Additionally, configuring cloud infrastructure correctly proved to be critical, as seen with the Dataproc cluster issue caused by networking constraints, demonstrating that even small configuration details can block progress. The use of Parquet format reinforced how important storage choices are for performance, as it enabled faster querying and more efficient storage compared to raw CSV files. Separating the pipeline into distinct layers (raw, processed, and warehouse) also improved organization, reproducibility, and reliability. Overall, the project reinforced the importance of thinking holistically about the entire data lifecycle while applying best practices in data engineering and cloud architecture.

9 Potential Next Steps

If more time were available, the following enhancements could be implemented:

Machine Learning

- Use BigQuery ML to build:
 - Discount prediction models
 - Price elasticity models
 - Stockout forecasting

Real-Time Data Pipeline

- Integrate Pub/Sub for streaming ingestion
- Enable near real-time pricing analytics

Advanced Analytics

- Time-series analysis of pricing trends
- Market segmentation analysis
- Product recommendation systems

Pipeline Improvements

- Automate pipeline with Cloud Composer (Airflow)
- Add monitoring and logging
- Implement CI/CD for pipeline deployment